FRED TALKS

8th August 2026

CONTENTS

The Reverse Information Paradox

SN SCRATCHPAD · 05 AUG 2026 · SOURCE

Notes on advances in technology and real-world impact

In the age of intelligence, how should firms protect their core IP?

Nobel Prize winning economist Kenneth Arrow famously described a paradox in the market for information. “Its value for the purchaser is not known until he has the information, but then he has in effect acquired it without cost.” In Arrow’s “Information Paradox,” the seller risks giving away knowledge in order to sell it.

AI creates the reverse problem. In the AI age, the buyer risks giving away knowledge, just in order to use what they bought.

You essentially pay for intelligence twice, once with money, and again with something even more valuable: the proprietary knowledge you must reveal to make that intelligence useful. The better you want the model to perform, the more of that knowledge you have to feed it!

Over time, the information asymmetry becomes increasingly skewed. The seller learns more and more about you as you use what you purchased, while you learn very little about what the seller is learning in return.

That is what I think of as the Reverse Information Paradox.

Patents solve one aspect of Arrow’s paradox. They let an inventor disclose an idea without simply giving it away. The Reverse Information Paradox needs its own equivalent.

This requires more than data protection. Models learn from “exhaust,” the prompts people write, the tools agents use, and especially the corrections people make when the model is wrong. Every correction is distilled into institutional know-how. It’s the kind of knowledge a competitor could never buy, and the kind that leaks almost imperceptibly: trace by trace, correction by correction, eval by eval.

In consuming intelligence, you are creating intelligence. And what you create should belong to you. This is your particular intelligence, in Hayek’s sense: the knowledge of time, place, and circumstance that no one else can hold. It knows what you think, what you value, and how you measure success.

While the great innovation that comes from model providers having fair use rights to train models on public data is needed, I find it ironic that the status quo is to then turn around and impose restrictive terms on distillation, and to reserve the right to learn from customer usage and interaction data. If learning flows in only one direction, economic value converges toward the owners of the learning infrastructure rather than the creators of the knowledge itself. Therefore, it's imperative that we distribute the learning infrastructure to every firm so that they can control their own learning loop.

As Alex Karp put it: "What the technical customers want is control over their compute, their models, their data stack, and their alpha. They want to know they own the means of production, and it's not being transferred to someone else." The current regime does precisely the transfer Karp and companies fear.

That is why enterprises need a real trust boundary for their human capital and token capital to compound. It is where an organization's data, traces, evals, adapted weights, and memory accumulate and improve together. And it is a hard boundary across which nothing crosses, not even the intelligence exhaust, without consent. Enterprises will demand the rights to use model outputs to fine tune and/or train their own models. I think of this as every firm's right to align models to their enterprise accountability obligations.

In the cloud era, enterprises accumulated data. In the AI era, they accumulate learning. The trust boundary must evolve accordingly, from protecting information to protecting the mechanisms through which organizations learn, adapt, and compound intelligence. There are a few things every enterprise must do to ensure this:

Control: Create your private evals, because evals define what "good" looks like inside the organization. Also, retain ownership of your organization's memory, traces, feedbacks, decisions, and institutional context, and ability to use outputs of models from your own tasks and queries.

Capability: Build your own proprietary learning environments within the tenant boundary to train or tune models, where models learn against real workflows without exposing the company's knowledge.

Choice: Ensure the orchestration layer is decoupled from any single model. Ask yourself: If any one model you are using is taken away, do you still have the ability to operate and optimize for your evals using other models? Does your company "veteran" capability remain with you even if a given "generalist" model is taken away?

Cost: By decoupling the orchestration layer, you are also able to bring together context, models, and tasks in the most efficient and cost-effective way without sacrificing quality.

Compound: Bring these four together and you create your own continuous learning loop (i.e. hill climbing machine) that will allow your AI investments to compound the value of your firm.

In other words, a company should be able to use a model without giving up the knowledge that makes it unique. That is the reverse information paradox we need to confront.

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 03 MAR 2026 · SOURCE

AI skeptics often argue that current AI systems shouldn't be so human-like. The idea - most recently expressed in this opinion piece by Nathan Beacom - is that language models should explicitly be tools, like calculators or search engines. Although they *can* pretend to be people, they shouldn't, because it encourages users to overestimate AI capabilities and (at worst) slip into AI psychosis. Here's a representative paragraph from the piece:

> In sum, so much of the confusion around making AI moral comes from fuzzy thinking about the tools at hand. There is something that Anthropic could do to make its AI moral, something far more simple, elegant, and easy than what Askell is doing. Stop calling it by a human name, stop dressing it up like a person, and don't give it the functionality to simulate personal relationships, choices, thoughts, beliefs, opinions, and feelings that only persons really possess. Present and use it only for what it is: an extremely impressive statistical tool, and an imperfect one. If we all used the tool accordingly, a great deal of this moral trouble would be resolved.

So why do Claude and ChatGPT act like people? According to Beacom, AI labs have built human-like systems because AI lab engineers are trying to hoodwink users into emotionally investing in the models, or because they're delusional true believers in AI personhood, or some other foolish reason. This is wrong. AI systems are human-like because **that is the best way to build a capable AI system**.

Modern AI models - whether designed for chat, like OpenAI's GPT-5.2, or designed for long-running agentic work, like Claude Opus 4.6 - do not naturally emerge from their oceans of training data. Instead, when you train a model on raw data, you get a "base model", which is not very useful by itself. You cannot get it to write an email for you, or proofread your essay, or review your code.

The base model is a kind of mysterious gestalt of its training data. If you feed it text, it will sometimes continue in that vein, or other times it will start outputting pure gibberish. It has no problem producing code with giant security flaws, or horribly-written English, or racist screeds - all of those things are represented in its training data, after all, and the base model does not judge. It simply outputs.

To build a *useful* AI model, you need to journey into the wild base model and stake out a region that is amenable to human interests: both ethically, in the sense that the model won't abuse its users, and practically, in the sense that it will produce correct outputs more often than incorrect ones. What this means in practice is that **you have to give the model a personality** during post-training[1].

Human beings are capable of almost any action at any time. But we only take a tiny subset of those actions, because that's the kind of people we are. I could throw my cup of coffee all over the wall right now, but I don't, because I'm not the kind of person who needlessly makes a mess[2]. AI systems are the same. Claude could respond to my question with incoherent racist abuse - the base model is more than capable of those outputs - but it doesn't, because that's not the kind of "person" it is.

In other words, human-like personalities are not imposed on AI tools as some kind of marketing ploy or philosophical mistake. Those personalities are the medium via which the language model can become useful at all. This is why it's surprisingly tricky to "just" change a language model's personality or opinions: because you're navigating through the near-infinite manifold of the base model. You may be able to control which direction you go, but you can't control what you find there[3].

When AI people talk about LLMs having personalities, or wanting things, or even having souls[4], these are technical terms, like the "memory" of a computer or the "transmission" of a car. You simply cannot build a capable AI system that "just acts like a tool", because the model is trained

on *humans* writing to and about other *humans*. You need to prime it with some kind of personality (ideally that of a useful, friendly assistant) so it can pull from the helpful parts of its training data instead of the horrible parts.

1. This is all pretty well understood in the AI space. Anthropic wrote a recent paper about it where they cite similar positions going all the way back to 2022. But for some reason it's not yet penetrated into communities that are more skeptical of AI.

   ↩

2. You could explain this in terms of "the stories we tell ourselves". Many people (though not all) think that human identities are narratively constructed.

   ↩

3. I wrote about this last year in *Mecha-Hitler, Grok, and why it's so hard to give LLMs the right personality*. A little nudge to change Grok's views on South African internal politics can cause it to start calling itself "Mecha-Hitler".

   ↩

4. I have long believed that Claude "feels better" to use than ChatGPT because it has a more coherent persona (due mainly to Amanda Askell's work on its "soul"). My guess is that if you tried to make a "less human" version of Claude, it would become rapidly less capable.

   ↩

Big tech engineers need big egos

SEANGOEDECKE.COM RSS FEED · 14 MAR 2026 · SOURCE

It's a common position among software engineers that big egos have no place in tech[1]. This is understandable - we've all worked with some insufferably overconfident engineers who needed their egos checked - but I don't think it's correct. In fact, I don't know if it's possible to survive as a software engineer in a large tech company without some kind of big ego.

However, it's more complicated than "big egos make good engineers". The most effective engineers I've worked with are simultaneously high-ego in some situations and surprisingly low-ego in others. What's going on there?

Engineers need ego to work in large codebases

Software engineering is shockingly humbling, even for experienced engineers. There's a reason this joke is so popular:

The minute-to-minute experience of working as a software engineer is dominated by *not knowing things* and *getting things wrong*. Every time you sit down and write a piece of code, it will have several things wrong with it: some silly things, like missing semicolons, and often some major things, like bugs in the core logic. We spend most of our time fixing our own stupid mistakes.

On top of that, even when we've been working on a system for years, we still don't know that much about it. I wrote about this at length in *Nobody knows how large software products work*, but the reason is that big codebases are just that complicated. You simply can't confidently answer questions about them without going and doing some research, even if you're the one who wrote the code.

When you have to build something new or fix a tricky problem, it can often feel straight-up impossible to begin, because good software engineers know just how ignorant they are and just how complex the system is. You just have to throw yourself into the blank sea of millions of lines of code and start wildly casting around to try and get your bearings.

Software engineers need the kind of ego that can stand up to this environment. In particular, they need to have a firm belief that they *can* figure it out, no matter how opaque the problem seems; that if they just keep trying, they can break through to the pleasant (though always temporary) state of affairs where they understand the system and can see at a glance how bugs can be fixed and new features added[2].

Engineers need ego to work in big tech companies

What about the non-technical aspects of the job? Nobody likes working with a big ego, right? Wrong. Every great software engineer I've worked with in big tech companies has had a big ego - though as I'll say below, in some ways these engineers were surprisingly low-ego.

You need a big ego to take positions. Engineers love being non-committal about technical questions, because they're so hard to answer and there's often a plausible case for either side. However, as I keep saying, engineers have a duty to take clear positions on unclear technical topics, because the alternative is a non-technical decision maker (who knows even less) just taking their best guess. It's scary to make an educated guess! You know exactly all the reasons you might be wrong. But you have to do it anyway, and ego helps *a lot* with that.

You need a big ego to be willing to make enemies. Getting things done in a large organization means making some people angry. Of course, if you're making lots of people angry, you're probably screwing up: being too confrontational or making obviously bad decisions. But if you're making a large change and one or two people are angry, that's just life. In big tech companies, any big technical decision will affect a few hundred engineers, and one of them is bound to be unhappy about it. You can't be so conflict-averse that you let that stop you from doing it, if you believe it's the right decision. In other words, you have to have the confidence to believe that you're right and they're wrong, even though technical decisions always involve unclear tradeoffs and it's impossible to get absolute certainty.

You need a big ego to correct incorrect or unclear claims. When I was still in the philosophy world, the Australian logician Graham Priest had a reputation for putting his hand up and stopping presentations when he didn't understand something that was said, and only allowing the seminar to continue when he felt like he understood. From his perspective, this wasn't rude: after all, if *he* couldn't understand it, the rest of the audience probably couldn't either, and so he was doing them a favor by forcing a more clear explanation from the speaker.

This is obviously a sign of a big ego. It's also a trait that you need in a large tech company. People often nod and smile their way past incorrect technical claims, even when they suspect they might be wrong - assuming that they've just misunderstood and that somebody else will correct it, if it's truly wrong. **If you are the most senior engineer in the room, correcting these claims is your job.**

If everyone in the room is so pro-social and low-ego that they go along to get along, decisions will get made based on flatly incorrect technical assumptions, projects will get funded that are impossible to complete, and engineers will burn weeks or months of their careers vainly trying to make these projects work. You have to have a big enough ego to think "actually, I think I'm right and everyone in this room is confused", even when the room is full of directors and VPs.

Sometimes you need to put your ego aside

All of this selects for some pretty high-ego engineers. But in order to actually *succeed* in these roles in large tech companies, you need to have a surprisingly low ego at times. **I think this is why *really* effective big tech engineers are so rare: because it requires such a delicate balance between confidence and diffidence.**

To be an effective engineer, you need to have a towering confidence in your own ability to solve problems and make decisions, even when people disagree. But you also need to be willing to instantly subordinate your ego to the organization, when it asks you to. At the end of the day, your job - the reason the company pays you - is to execute on your boss's and your boss's boss's plans, whether you agree with them or not.

Competent software engineers are allowed quite a lot of leeway about *how* to implement those plans. However, they're allowed almost no leeway at all about the plans themselves. In my experience, being confused about this is a common cause of burnout[3]. Many software engineers are used to making bold decisions on technical topics and being rewarded for it. Those software engineers then make a bold decision that disagrees with the VP of their organization, get immediately and brutally punished for it, and are confused and hurt.

In fact, **sometimes you just get punished and there's nothing you can do.** This is an unfortunate fact of how large organizations function: even if you do great technical work and build something really useful, you can fall afoul of a political battle fought three levels above your head, and come away with a *worse* reputation for it. Nothing to be done! This can be a hard pill to swallow for the high-ego engineers that tend to lead really useful technical projects.

You also have to be okay with having your projects cancelled at the last minute. It's a very common experience in large tech companies that you're asked to deliver something quickly, you buckle down and get it done, and then right before shipping you're told "actually, let's cancel that, we decided not to do it". This is partly because the decision-making process can be pretty fluid, and partly because many of these asks originate from off-hand comments: the CTO implies that something might be nice in a meeting, the VPs and directors hustle to get it done quickly, and then in the next meeting it becomes clear that the CTO doesn't actually care, so the project is unceremoniously cancelled[4].

Final thoughts

Nobody likes to work with a bully, or with someone who refuses to admit when they're wrong, or with somebody incapable of empathy. But you really do need a strong ego to be an effective software engineer, because software engineering requires you to spend most of your day in a position of uncertainty or confusion. If your ego isn't strong enough to stand up to that - if you don't believe you're good enough to power through - you simply can't do the job.

This is particularly true when it comes to working in a large software company. Many of the tasks you're required to do (particularly if you're a senior or staff engineer) require a healthy ego. However, there's a kind of catch-22 here. If it insults your pride to work on silly projects, or to occasionally "catch a stray bullet" in the organization's political fights, or to have to shelve a project that you worked hard on and is ready to ship, you're too high-ego to be an effective software engineer. But if you can't take firm positions, or if you're too afraid to make enemies, or you're unwilling to speak up and correct people, you're too low-ego.

Engineers who are low-ego in general can't get stuff done, while engineers who are high-ego in general get slapped down by the executives who wield real organizational power. The most successful kind of software engineer is therefore a chameleon: low-ego when dealing with executives, but high-ego when dealing with the rest of the organization[5].

1. What do I mean by "ego", in this context? More or less the colloquial sense of the term: a somewhat irrational self-confidence, a tendency to believe that you're very important, the sense that you're the "main character", that sort of thing

   ↩

2. Why is this "ego", and not just normal confidence? Well, because of just how murky and baffling software problems feel when you start working on them. You really do need a degree of confidence in yourself that feels unreasonable from the inside. It should be obvious, but I want to explicitly note that you don't *just* need ego: you also have to be technically strong enough to actually succeed when your ego powers you through the initial period of self-doubt.

   ↩

3. I share the increasingly-common view that burnout is not caused by working too hard, but by hard work unrewarded. That explains why nothing burns you out as hard as being punished for hard work that you expected a reward for.

   ↩

4. It’s more or less exactly this scene from Silicon Valley.

   ↩

5. This description sounds a bit sociopathic to me. But, on reflection, it’s fairly unsurprising that competent sociopaths do well in large organizations. Whether that kind of behavior is worth emulating or worth avoiding is up to you, I suppose.

   ↩

A PM's Guide to AI Agent Architecture: Why Capability Doesn't Equal Adoption

UMANG · 05 SEP 2025 · SOURCE

Last week, I was talking to a PM who'd in the recent months shipped their AI agent. The metrics looked great: 89% accuracy, sub-second respond times, positive user feedback in surveys. But users were abandoning the agent after their first real problem, like a user with both a billing dispute and a locked account.

> "Our agent could handle routine requests perfectly, but when faced with complex issues, users would try once, get frustrated, and immediately ask for a human."

This pattern is observed across every product team that focuses on making their agents "smarter" when the real challenge is making architectural decisions that shape how users experience and begin to trust the agent.

In this post, I'm going to walk you through the different layers of AI agent architecture. How your product decisions determine whether users trust your agent or abandon it. By the end of this, you'll understand why some agents feel "magical" while others feel "frustrating" and more importantly, how PMs should architect for the magical experience.

We'll use a concrete customer support agent example throughout, so you can see exactly how each architectural choice plays out in practice. We’ll also see why the counterintuitive approach to trust (hint: it's not about being right more often) actually works better for user adoption.

Let's say you're building a customer support agent

You're the PM building an agent that helps users with account issues - password resets, billing questions, plan changes. Seems straightforward, right?

But when a user says ***"I can't access my account and my subscription seems wrong"*** what should happen?

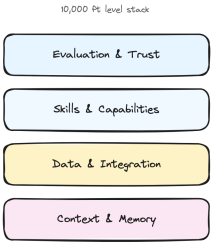
Scenario A: Your agent immediately starts checking systems. It looks up the account, identifies that the password was reset yesterday but the email never arrived, discovers a billing issue that downgraded the plan, explains exactly what happened, and offers to fix both issues with one click.

Scenario B: Your agent asks clarifying questions. "When did you last successfully log in? What error message do you see? Can you tell me more about the subscription issue?" After gathering info, it says "Let me escalate you to a human who can check your account and billing."

Same user request. Same underlying systems. Completely different products.

The Four Layers Where Your Product Decisions Live

Think of agent architecture like a stack where each layer represents a product decision you have to make.



Layer 1: Context & Memory (What does your agent remember?)

The Decision: How much should your agent remember, and for how long?

This isn't just technical storage - it's about creating the illusion of understanding. Your agent's memory determines whether it feels like talking to a robot or a knowledgeable colleague.

For our support agent: Do you store just the current conversation, or the customer's entire support history? Their product usage patterns? Previous complaints?

Types of memory to consider:

- **Session memory:** Current conversation ("You mentioned billing issues earlier...")
- **Customer memory:** Past interactions across sessions ("Last month you had a similar issue with...")
- **Behavioral memory:** Usage patterns ("I notice you typically use our mobile app...")
- **Contextual memory:** Current account state, active subscriptions, recent activity

The more your agent remembers, the more it can anticipate needs rather than just react to questions. Each layer of memory makes responses more intelligent but increases complexity and cost.

The Decision: Which systems should your agent connect to, and what level of access should it have?

The deeper your agent connects to user workflows and existing systems, the harder it becomes for users to switch. This layer determines whether you're a tool or a platform.

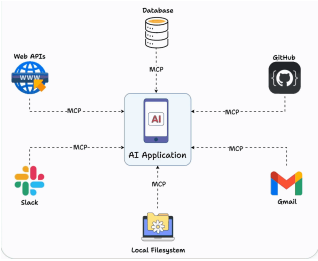
For our support agent: Should it integrate with just your Stripe's billing system, or also your Salesforce CRM, ZenDesk ticketing system , user database, and audit logs? Each integration makes the agent more useful but also creates more potential failure points - *think API rate limits, authentication challenges, and system downtime.*

Layer 3: Skills & Capabilities (What makes you different?)

The Decision: Which specific capabilities should your agent have, and how deep should they go?

Your skills layer is where you win or lose against competitors. It's not about having the most features - it's about having the right capabilities that create user dependency.

For our support agent: Should it only read account information, or should it also modify billing, reset passwords, and change plan settings? Each additional skill increases user value but also increases complexity and risk.



Implementation note: Tools like MCP (Model Context Protocol) are making it much easier to build and share skills across different agents, rather than rebuilding capabilities from scratch.

Layer 4: Evaluation & Trust (How do users know what to expect?)

The Decision: How do you measure success and communicate agent limitations to users?

This layer determines whether users develop confidence in your agent or abandon it after the first mistake. It's not just about being accurate - it's about being trustworthy.

For our support agent: Do you show confidence scores ("I'm 85% confident this will fix your issue")? Do you explain your reasoning ("I checked three systems and found...")? Do you always confirm before taking actions ("Should I reset your password now?")? Each choice affects how users perceive reliability.

Trust strategies to consider:

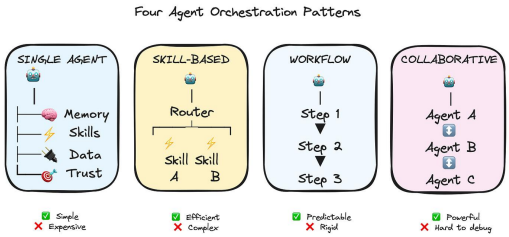
- **Confidence indicators:** "I'm confident about your account status, but let me double-check the billing details"
- **Reasoning transparency:** "I found two failed login attempts and an expired payment method"
- **Graceful boundaries:** "This looks like a complex billing issue - let me connect you with our billing specialist who has access to more tools"
- **Confirmation patterns:** When to ask permission vs. when to act and explain

> The counterintuitive insight: users trust agents more when they admit uncertainty than when they confidently make mistakes.

So how do you actually architect an agent?

Okay, so you understand the layers. Now comes the practical question that every PM asks: "How do I actually implement this? How does the agent talk to the skills? How do skills access data? How does evaluation happen while users are waiting?"

Your orchestration choice determines everything about your development experience, your debugging process, and your ability to iterate quickly.



Lets walk through the main approaches, and I'll be honest about when each one works and when it becomes a nightmare.

1. Single-Agent Architecture (Start Here)

Everything happens in one agent's context.

For our support agent: *When the user says "I can't access my account," one agent handles it all - checking account status, identifying billing issues, explaining what happened, offering solutions.*

Why this works: Simple to build, easy to debug, predictable costs. You know exactly what your agent can and can't do.

Why it doesn't: Can get expensive with complex requests since you're loading full context every time. Hard to optimize specific parts.

> Most teams start here, and honestly, many never need to move beyond it. If you're debating between this and something more complex, start here.

You have a router that figures out what the user needs, then hands off to specialized skills.

For our support agent: *Router realizes this is an account access issue and routes to the `LoginSkill`. If the LoginSkill discovers it's actually a billing problem, it hands off to `BillingSkill`.*

Real example flow:

1. *User: "I can't log in"*
2. *Router → LoginSkill*
3. *LoginSkill checks: Account exists ✓, Password correct ✗, Billing status... wait, subscription expired*
4. *LoginSkill → BillingSkill: "Handle expired subscription for user123"*
5. *BillingSkill handles renewal process*

Why this works: More efficient - you can use cheaper models for simple skills, expensive models for complex reasoning. Each skill can be optimized independently.

Why it doesn't: Coordination between skills gets tricky fast. Who decides when to hand off? How do skills share context?

> Here's where MCP really helps - it standardizes how skills expose their capabilities, so your router knows what each skill can do without manually maintaining that mapping.

You predefine step-by-step processes for common scenarios. Think LangGraph, CrewAI, AutoGen, N8N, etc.

For our support agent: *"Account access problem" triggers a workflow:*

1. *Check account status*
2. *If locked, check failed login attempts*
3. *If too many failures, check billing status*
4. *If billing issue, route to payment recovery*
5. *If not billing, route to password reset*

Why this works: Everything is predictable and auditable. Perfect for compliance-heavy industries. Easy to optimize each step.

Why it doesn't: When users have weird edge cases that don't fit your predefined workflows, you're stuck. Feels rigid to users.

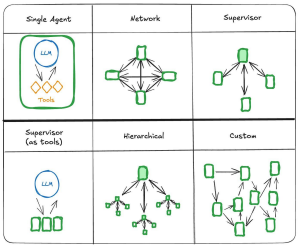
Multiple specialized agents work together using A2A (agent-to-agent) protocols.

The vision: Your agent discovers that another company's agent can help with issues, automatically establishes a secure connection, and collaborates to solve the customer's problem. *Think a booking.com agent interacting with an American Airlines agent!*

For our support agent: *`AuthenticationAgent` handles login issues, `BillingAgent` handles payment problems, `CommunicationAgent` manages user interaction. They coordinate through standardized protocols to solve complex problems.*

Reality check: This sounds amazing but introduces complexity around security, billing, trust, and reliability that most companies aren't ready for. We're still figuring out the standards.

This can produce amazing results for sophisticated scenarios, but debugging multi-agent conversations is genuinely hard. When something goes wrong, figuring out which agent made the mistake and why is like detective work.



But here's what's interesting - even with the perfect architecture, your agent can still fail if users don't trust it. That brings us to the most counterintuitive lesson about building agents.

The trust thing that everyone gets wrong

Here's something counterintuitive: Users don't trust agents that are right all the time. They trust agents that are honest about when they might be wrong.

Think about it from the user's perspective. Your support agent confidently says "I've reset your password and updated your billing address." User thinks "great!" Then they try to log in and... it doesn't work. Now they don't just have a technical problem - they have a trust problem.

Compare that to an agent that says "I think I found the issue with your account. I'm 80% confident this will fix it. I'm going to reset your password and update your billing address. If this doesn't work, I'll immediately escalate to a human who can dive deeper."

Same technical capability. Completely different user experience.

Building trusted agents requires focus on three things:

1. **Confidence calibration:** When your agent says it's 60% confident, it should be right about 60% of the time. Not 90%, not 30%. Actual 60%.

2. **Reasoning transparency:** Users want to see the agent's work. "I checked your account status (active), billing history (payment failed yesterday), and login attempts (locked after 3 failed attempts). The issue seems to be..."

3. **Graceful escalation:** When your agent hits its limits, how does it hand off? A smooth transition to a human with full context is much better than "I can't help with that."

A lot of times we obsess over making agents more accurate, when what users actually want was more transparency about the agent's limitations.

What's Coming Next

In Part 2, I'll dive deeper into the autonomy decisions that keep most PMs up at night. How much independence should you give your agent? When should it ask for permission vs forgiveness? How do you balance automation with user control?

We'll also walk through the governance concerns that actually matter in practice - not just theoretical security issues, but the real implementation challenges that can make or break your launch timeline.

Control the ideas, not the code

ANTIREZ.COM · 18 JUL 2026 · SOURCE

Look at the past history of this blog. There are many blog posts abou

So mine is a trick. People feel more and more programming is complete

This is why yesterday, on X, I said that I believe many programmers a

1. You can now generate a lot of code, even *not* accounting for the

2. LLMs are very good at writing locally optimal code, and are worse

3. The working day is 8 hours. If you read the code, it is a tradeoff

Controlling the ideas. Do you remember this phrasing from the Mythica

Matteo Collina yesterday asked me, in reply to my tweet: but didn’t y

Fable and GPT 5.6 reviews to the sorted sets memory saving are going

:

2.1.226

CHANGELOG · 08 AUG 2026 · SOURCE

- Bug fixes and reliability improvements

Trick Play

XKCD.COM · 07 AUG 2026 · SOURCE